

Data Preprocessing & Visualization

Session 2: (Pandas)

Agenda:

1	Introduction to Pandas
2	Data Manipulation with Pandas
3	Practical Examples

Introduction to Pandas

Introduction to Pandas

What is Pandas?

Pandas is an open-source data analysis and data manipulation library built on top of Numpy. It provides high-level data structures and functions designed to make data analysis fast and easy in Python.

Pandas is widely used in data science, finance, statistics, and many other fields for tasks like data cleaning, transformation, and analysis.

Introduction to Pandas

Key Features of Pandas

- **Series and DataFrames:** Pandas introduces two primary data structures: Series (one-dimensional) and DataFrame (two-dimensional), which are used to store and manipulate data.
- **Data Alignment:** Pandas automatically aligns data by its labels, making it easy to perform arithmetic operations on data sets with different indexes.
- **Handling Missing Data:** Pandas provides methods for detecting, filling, and dropping missing data, which is essential for cleaning datasets.
- **Powerful Data Grouping and Aggregation:** Pandas supports grouping data, aggregating results, and applying custom functions using the `groupby()` functionality.
- **Flexible Data Reading/Writing:** Pandas can read from and write to various file formats, including CSV, Excel, SQL databases, and more.

Data Manipulation with Pandas

Data manipulation is a core aspect of data analysis. Pandas provides various tools and functions to manipulate data efficiently.



Creating Pandas Data Structures

Creating a Pandas Series

```
import pandas as pd

# Creating a Series from a list
series = pd.Series([10, 20, 30, 40, 50])
print(series)
```

```
0    10
1    20
2    30
3    40
4    50
dtype: int64
```

A Series is a one-dimensional labeled array capable of holding any data type. It is similar to a column in a spreadsheet.

Creating Pandas Data Structures

Creating a DataFrame from a Dictionary

```
[2] data = {'Name': ['Alice', 'Bob', 'Charlie'],  
          'Age': [25, 30, 35],  
          'City': ['New York', 'Los Angeles', 'Chicago']}  
df = pd.DataFrame(data)  
print(df)
```

```
↔
```

	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
2	Charlie	35	Chicago

A DataFrame is a two-dimensional labeled data structure with columns of potentially different types. It is similar to a table in a database or a data frame in R.

Viewing and Inspecting Data

Viewing the First Few Rows

```
df.head()
```



	Name	Age	City
0	Alice	25	New York
1	Bob	30	Los Angeles
2	Charlie	35	Chicago

The `head()` method returns the first five rows of the DataFrame. This is useful for quickly examining the top of a dataset.

Viewing and Inspecting Data

Getting DataFrame Information

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 3 entries, 0 to 2  
Data columns (total 3 columns):  
#   Column  Non-Null Count  Dtype  
---  ---      -  
0   Name    3 non-null      object  
1   Age     3 non-null      int64  
2   City    3 non-null      object  
dtypes: int64(1), object(2)  
memory usage: 200.0+ bytes
```

The `info()` method provides a summary of the DataFrame, including the number of non-null entries and the data type of each column.

Viewing and Inspecting Data

Describing Statistical Information

```
df.describe()
```



	Age
count	3.0
mean	30.0
std	5.0
min	25.0
25%	27.5
50%	30.0
75%	32.5
max	35.0

The describe() method generates descriptive statistics, such as mean, standard deviation, min, and max, for numerical columns.

Selecting and Filtering Data

Selecting Columns

```
print(df['Name'])
```

```
0    Alice  
1     Bob  
2   Charlie  
Name: Name, dtype: object
```

You can select a single column from a DataFrame using the column name. This returns a Series object.

Selecting and Filtering Data

Filtering Rows Based on Condition

```
[7] filtered_df = df[df['Age'] > 30]  
     print(filtered_df)
```

	Name	Age	City
2	Charlie	35	Chicago

You can filter rows based on a condition. Here, only rows where the 'Age' column is greater than 30 are selected.

Selecting and Filtering Data

Selecting Specific Rows and Columns

```
selected_data = df.loc[0:1, ['Name', 'City']]  
print(selected_data)
```

```
↔
```

	Name	City
0	Alice	New York
1	Bob	Los Angeles

The `loc[]` method is used for label-based indexing to select specific rows and columns. Here, rows with indices 0 and 1 and columns 'Name' and 'City' are selected.

Modifying Data

Adding a New Column

```
df['Salary'] = [70000, 80000, 90000]  
print(df)
```

	Name	Age	City	Salary
0	Alice	25	New York	70000
1	Bob	30	Los Angeles	80000
2	Charlie	35	Chicago	90000

A new column 'Salary' is added to the DataFrame with specified values. This is a common operation to add calculated fields or new data.

Modifying Data

Updating Column Values

```
df['Age'] = df['Age'] + 1  
print(df)
```

	Name	Age	City	Salary
0	Alice	26	New York	70000
1	Bob	31	Los Angeles	80000
2	Charlie	36	Chicago	90000

This updates the values in the 'Age' column by adding 1 to each value. Pandas allows vectorized operations, making this efficient.

Modifying Data

Dropping Columns

```
df = df.drop('City', axis=1)  
print(df)
```

	Name	Age	Salary
0	Alice	26	70000
1	Bob	31	80000
2	Charlie	36	90000

The `drop()` method removes the 'City' column from the DataFrame. The `axis=1` argument specifies that a column is being dropped.

Handling Missing Data

Detecting Missing Values

```
df_with_nan = pd.DataFrame({'A': [1, 2, None], 'B': [4, None, 6]})  
print(df_with_nan.isnull())
```

```
   A      B  
0 False False  
1 False  True  
2  True False
```

The `isnull()` method detects missing values in the DataFrame and returns a DataFrame of the same shape with True or False values.

Handling Missing Data

Filling Missing Values

```
filled_df = df_with_nan.fillna(0)  
print(filled_df)
```

```
   A  B  
0  1.0  4.0  
1  2.0  0.0  
2  0.0  6.0
```

The `fillna()` method replaces missing values with a specified value, such as 0. This is useful for cleaning datasets before analysis.

Handling Missing Data

Dropping Rows with Missing Values

```
▶ cleaned_df = df_with_nan.dropna()  
print(cleaned_df)
```

```
↔  
   A  B  
0  1.0 4.0
```

The `dropna()` method removes rows with any missing values. This can be useful when working with datasets where missing values cannot be tolerated.

Grouping and Aggregation

Grouping Data and Calculating Mean

```
[15] df = pd.DataFrame({'Department': ['HR', 'IT', 'HR', 'IT'],  
                      'Salary': [50000, 60000, 45000, 80000]})  
grouped = df.groupby('Department').mean()  
print(grouped)
```



	Salary
Department	
HR	47500.0
IT	70000.0

The `groupby()` method groups the data by a specified column and then calculates the mean salary for each department. This is commonly used for summarizing data.

Grouping and Aggregation

Applying Custom Functions with apply()

```
def double_salary(x):  
    return x * 2  
  
df['Double Salary'] = df['Salary'].apply(double_salary)  
print(df)
```

	Department	Salary	Double Salary
0	HR	50000	100000
1	IT	60000	120000
2	HR	45000	90000
3	IT	80000	160000

The apply() method is used to apply a custom function to each element in a column. Here, it doubles the salary for each employee.

Practical Examples

Practical Examples

We'll load this CSV file into a Pandas DataFrame and perform various data manipulation tasks

```
import pandas as pd

# Load the CSV file into a Pandas DataFrame
df = pd.read_csv('employees.csv')
```


Practical Examples

Viewing the First Few Rows of the DataFrame

```
print("First 5 Rows:\n")  
df.head()
```

First 5 Rows:

	ID	Name	Department	Age	Salary	Joining_Date
0	1	John Doe	HR	28	50000	2020-05-21
1	2	Jane Smith	IT	34	75000	2019-06-12
2	3	Bob Johnson	Finance	45	85000	2018-07-15
3	4	Alice White	IT	29	72000	2021-01-04
4	5	Charlie Brown	Marketing	32	68000	2019-11-30

The head() method displays the first five rows of the DataFrame, which is useful for getting a quick look at the data.

Practical Examples

Displaying Basic Information About the DataFrame

```
print("DataFrame Information:\n")  
df.info()
```



DataFrame Information:

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 10 entries, 0 to 9  
Data columns (total 6 columns):  
#   Column          Non-Null Count  Dtype  
---  ---  
0   ID              10 non-null    int64  
1   Name            10 non-null    object  
2   Department      10 non-null    object  
3   Age             10 non-null    int64  
4   Salary          10 non-null    int64  
5   Joining_Date    10 non-null    object  
dtypes: int64(3), object(3)  
memory usage: 608.0+ bytes
```

The info() method provides a summary of the DataFrame, including the number of non-null values, data types, and memory usage.

Practical Examples

Calculating Basic Statistics for Numerical Columns

```
print("Statistical Summary:\n")  
df.describe()
```

Statistical Summary:

	ID	Age	Salary
count	10.00000	10.000000	10.000000
mean	5.50000	35.300000	68300.000000
std	3.02765	7.958922	13106.826042
min	1.00000	26.000000	50000.000000
25%	3.25000	29.250000	57250.000000
50%	5.50000	33.000000	69000.000000
75%	7.75000	40.250000	74250.000000
max	10.00000	50.000000	90000.000000

The describe() method generates descriptive statistics such as mean, standard deviation, and percentiles for numerical columns.

Practical Examples

Filtering Employees by Department

```
▶ it_employees = df[df['Department'] == 'IT']  
print("IT Department Employees:\n")  
it_employees.head()
```

⇒ IT Department Employees:

	ID	Name	Department	Age	Salary	Joining_Date
1	2	Jane Smith	IT	34	75000	2019-06-12
3	4	Alice White	IT	29	72000	2021-01-04
7	8	Linda Taylor	IT	26	64000	2022-02-10

This code filters the DataFrame to show only employees who work in the IT department using a condition on the 'Department' column.

Practical Examples

Calculating the Average Salary by Department

```
average_salary_by_dept = df.groupby('Department')['Salary'].mean()  
print("Average Salary by Department:\n")  
average_salary_by_dept.head()
```

➡ Average Salary by Department:

Salary	
Department	
Finance	87500.000000
HR	53000.000000
IT	70333.333333
Marketing	69000.000000

dtype: float64

The `groupby()` method groups the data by the 'Department' column and calculates the mean salary for each department using `mean()`.

Practical Examples

Adding a New Column for Salary After a 5% Raise

```
df['Salary_After_Raise'] = df['Salary'] * 1.05  
print("DataFrame with Salary After Raise:\n" )  
df.head()
```

DataFrame with Salary After Raise:

	ID	Name	Department	Age	Salary	Joining_Date	Salary_After_Raise
0	1	John Doe	HR	28	50000	2020-05-21	52500.0
1	2	Jane Smith	IT	34	75000	2019-06-12	78750.0
2	3	Bob Johnson	Finance	45	85000	2018-07-15	89250.0
3	4	Alice White	IT	29	72000	2021-01-04	75600.0
4	5	Charlie Brown	Marketing	32	68000	2019-11-30	71400.0

A new column is added to the DataFrame by multiplying the 'Salary' column by 1.05 to reflect a 5% salary increase.

Practical Examples

Sorting Employees by Age in Descending Order

```
sorted_by_age = df.sort_values(by='Age', ascending=False)  
print("Employees Sorted by Age (Descending):\n")  
sorted_by_age.head()
```

Employees Sorted by Age (Descending):

	ID	Name	Department	Age	Salary	Joining_Date	Salary_After_Raise
6	7	Mike Davis	Finance	50	90000	2016-08-19	94500.0
2	3	Bob Johnson	Finance	45	85000	2018-07-15	89250.0
5	6	Eva Green	HR	41	54000	2017-04-22	56700.0
8	9	James Wilson	Marketing	38	70000	2020-09-08	73500.0
1	2	Jane Smith	IT	34	75000	2019-06-12	78750.0

The `sort_values()` method sorts the DataFrame by the 'Age' column in descending order, showing the oldest employees first.

Practical Examples

Calculating the Number of Employees in Each

```
▶ employee_count_by_dept = df['Department'].value_counts()  
print("Number of Employees by Department:\n")  
employee_count_by_dept.head()
```

➡ Number of Employees by Department:

count	
Department	
HR	3
IT	3
Finance	2
Marketing	2

dtype: int64

The `value_counts()` method counts the number of occurrences of each unique value in the 'Department' column, showing how many employees are in each department.

Practical Examples

Finding Employees Who Joined After 2020

```
df['Joining_Date'] = pd.to_datetime(df['Joining_Date'])  
recent_joins = df[df['Joining_Date'] > '2020-01-01']  
print("Employees Joined After 2020:\n")  
recent_joins.head()
```

→ Employees Joined After 2020:

	ID	Name	Department	Age	Salary	Joining_Date	Salary_After_Raise
0	1	John Doe	HR	28	50000	2020-05-21	52500.0
3	4	Alice White	IT	29	72000	2021-01-04	75600.0
7	8	Linda Taylor	IT	26	64000	2022-02-10	67200.0
8	9	James Wilson	Marketing	38	70000	2020-09-08	73500.0
9	10	Sarah Adams	HR	30	55000	2021-03-18	57750.0

This converts the 'Joining_Date' column to datetime format and filters the DataFrame to find employees who joined after January 1, 2020.

Practical Examples

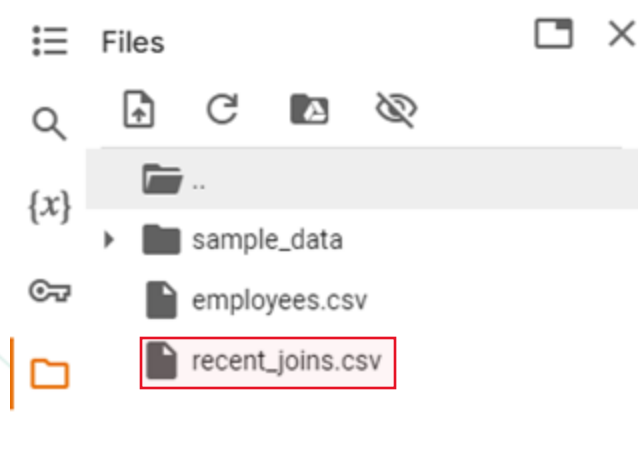
Saving the Filtered DataFrame to a New CSV File

```
recent_joins.to_csv('recent_joins.csv', index=False)  
print("Filtered data saved to recent_joins.csv.")
```

⇒ Filtered data saved to recent_joins.csv.

The `to_csv()` method saves the filtered DataFrame (`recent_joins`) to a new CSV file named `'recent_joins.csv'`. The `index=False` argument excludes the row index from the CSV file.

Summary



Let's Code!



Any Questions?



Quiz Time!

Thanks